

Tutorial 3 Worksheet

INFORMATION Tutorial format

Work in a group of four and select one facilitator for each activity. The facilitator invites contributions from all members, ensures that everyone builds understanding, monitors time, and checks that the group agrees on an approach before continuing.

EXERCISE 1 Input - Output Models

An engineering model maps one or more inputs to one or more outputs:

$$y = f_{\theta}(x)$$

Here, x is the input, y is the output and θ contains the parameters that determine how the model responds. The first two entries in the table below are examples.

System / model	Input, x	Output, y	Parameters, θ	How parameters are obtained	Type
Resistor $V = RI$	Current, I	Voltage, V	Resistance, R	Data sheet or measured V/I pairs	Deterministic
Rainfall classifier	Temperature, humidity, pressure, rainfall and wind	Rain tomorrow: Yes / No	Multi-Layer Perceptron (MLP) weights and biases	Learned from labelled historical data	Deterministic

- Complete the table with systems found in your engineering fields. Include at least one deterministic and one stochastic system.
- For each example, decide whether the parameters can be measured directly, obtained from physical knowledge, or learned from experimental data.
- Select one system from your table. Would you predict its output using a physical formula, a computer simulation or an AI model? Discuss why.

EXERCISE 2 Building a Rainfall Classifier

Your group will build a model that uses weather observations from one day to predict whether it will rain the following day. The output label is `RainTomorrow`, with the values `Yes` and `No`. This is a binary classification task.

a) Responsible use of generative AI

Before writing code, draw on the Week 2 discussion of responsible AI use, ethics and academic integrity. Identify one ethical and responsible way, and one unethical or irresponsible way, to use generative AI in this activity. Record the key features that distinguish them and explain why these differences matter for your learning and integrity.

Before you continue with 'b) Follow AI/ML Pipeline', discuss your proposed approach with your tutor.

b) Follow AI/ML Pipeline

In this exercise, we will follow the AI/ML pipeline introduced in the lecture and shown in Figure 1. Dividing the modelling process into defined stages helps ensure that important steps are not overlooked, decisions can be checked and reproduced, and the final model is evaluated fairly using unseen data.

In your ENGG2112 project, you will follow a similar process, from data cleaning & preprocessing through to model evaluation. However, you will work with an existing dataset and will not deploy or monitor the model.

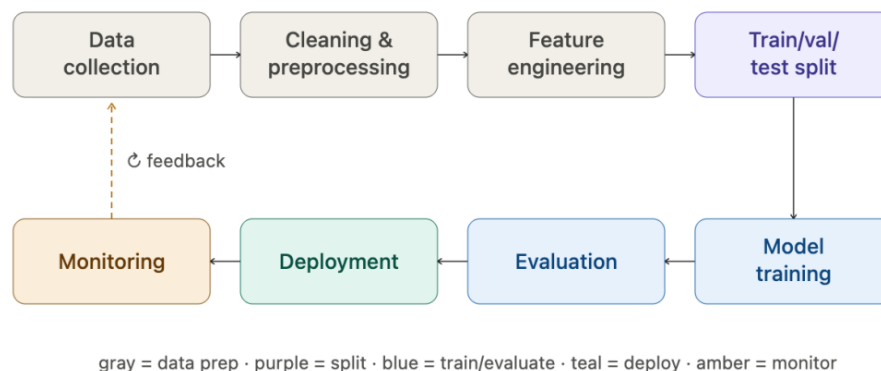


Figure 1. AI/ML pipeline introduced in the Week 3 lecture.

1. Data Collection

Hugging Face is an open platform for sharing machine-learning models and datasets. A dataset page provides the files and, where available, information about the dataset's contents and origin.

Dataset: [AiiN-aini/weatherAUS](#)

Download `weatherAUS.csv` through the ‘Files and versions’ tab, open and inspect it using a csv-viewer such as Excel, and explore the variable ranges shown on the Dataset Viewer on Hugging Face.

Create a Jupyter Notebook (.ipynb) and place weatherAUS.csv in the same folder. A Python file (.py) may also be used, but some commands below, such as `df.head()`, will not display automatically.

In the notebook, import pandas, NumPy and matplotlib, then load the CSV. Use the following pandas methods to explore it:

```
df.head()
df.shape
df.info()
df.isna().sum()
df['RainTomorrow'].value_counts(dropna=False)
```

Create a bar chart showing the number of Yes and No labels in RainTomorrow. Add a title and labelled axes.

2. Cleaning and Preprocessing

Numerical features are measured quantities represented by numbers. Categorical features represent groups or labels and normally require encoding before a model can use them. For this first model, use the numerical features below and leave Date, Location, the wind-direction columns and RainToday out.

```
features = ['MinTemp', 'MaxTemp', 'Rainfall', 'WindGustSpeed',
            'Humidity9am', 'Humidity3pm', 'Pressure9am',
            'Pressure3pm', 'Temp9am', 'Temp3pm']
```

Create a new DataFrame containing these features and RainTomorrow. Use dropna() to remove rows with missing values in this subset, and report how many rows remain. Convert RainTomorrow to 0 for No and 1 for Yes.

3. Feature Engineering

Create TemperatureRange = MaxTemp - MinTemp and add it to the feature list. Inspect a few values to confirm that the calculation is sensible.

Discuss other features that could be created from the available variables and how they might benefit the rainfall classifier.

4. Train/Test Split

Use train_test_split() to allocate 80% of the cleaned data to training and 20% to testing. This will produce the feature sets X_train and X_test and the corresponding label sets y_train and y_test. A validation set would normally be used to compare models or tune model settings, but it is omitted here because we are using one predefined model configuration. Set random_state=2112 to make the split reproducible. Set stratify=y to preserve approximately the same proportion of rainy and non-rainy samples in both sets. Print the sizes of the four resulting objects.

5. Standardisation – A preprocessing step

The features use different units and ranges. StandardScaler places them on comparable scales, which helps an MLP train reliably. Fit the scaler using only X_train, then use it to transform both X_train and X_test.

Although standardisation is a preprocessing step, it is performed after the train–test split because its parameters must be calculated using only the training data to prevent information from the test set influencing the model.

6. Model Training

Rainfall depends on interacting and threshold-like relationships between humidity, pressure, temperature and wind. A Multi-Layer Perceptron (MLP) can learn nonlinear combinations of these inputs rather than assuming a single linear relationship. It is used here because it connects to the lecture, but it is not necessarily the best-performing model for this dataset.

```
from sklearn.neural_network import MLPClassifier
model = MLPClassifier(hidden_layer_sizes=(10,),
                      max_iter = 300,
                      random_state = 2112)
```

This defines one hidden layer with 10 neurons, allows up to 300 training iterations and makes repeated runs reproducible. Use generative AI to help determine how to train the model using model.fit() and generate predictions using model.predict(). Store the test-set predictions as y_pred. Every group member should be able to explain the resulting code.

7. Evaluation

7.1 Extract TP, TN, FP and FN

Place the true and predicted labels in a pandas DataFrame:

```
results = pd.DataFrame({
    'actual': y_test.to_numpy(),
    'predicted': y_pred
})
results.head()
```

The following example counts true positives. Write equivalent expressions for TN, FP and FN.

```
TP = ((results['actual'] == 1) & (results['predicted'] == 1)).sum()
```

	Predicted rain (1)	Predicted no rain (0)
Actual rain (1)	TP =	FN =
Actual no rain (0)	FP =	TN =

Check that $TP + TN + FP + FN$ equals the number of test samples.

7.2 Calculate the performance metrics manually

Use TP, FN, FP and TN and the equations below to calculate each metric.

Metric	Equation	Your result
Accuracy	$(TP + TN) / (TP + TN + FP + FN)$	
Precision	$TP / (TP + FP)$	
Recall	$TP / (TP + FN)$	
F1	$2 \times \text{Precision} \times \text{Recall} / (\text{Precision} + \text{Recall})$	

7.3 Check the results using scikit-learn

Import and use the functions below. Compare their outputs with your manual calculations.

```
from sklearn.metrics import (confusion_matrix, accuracy_score,
                             precision_score, recall_score, f1_score)
```

For labels ordered as [0, 1], confusion_matrix() returns [[TN, FP], [FN, TP]]. If your values differ, first check the label order and your Boolean conditions.

7.4 Compare with a simple baseline

Use NumPy to create a baseline that predicts No for every test sample, then calculate its accuracy:

```
baseline_pred = np.zeros_like(y_test)
```

Compare the baseline accuracy with the MLP accuracy. A useful classifier should be judged against a reasonable baseline, not against zero performance.

7.5 Improving the model

The current MLP model should outperform the simple baseline, but its performance can be improved. Experiment with the model configuration by changing 'hidden_layer_sizes', or try another suitable classification model. Compare the resulting performance metrics with those of the original MLP.

Model selection and parameter tuning would normally use a separate validation set to avoid repeatedly making decisions based on the test data. No validation set is used here to keep this activity simple. Therefore, treat this comparison as an exploration rather than a reliable estimate of which model will perform best on new data.

8. Deployment, Monitoring and Feedback

You do not need to implement this step.

After evaluation, a suitable model could be deployed within an application to generate predictions from new weather observations. Its performance would then be monitored by comparing these predictions with the weather that actually occurred.

Monitoring may reveal changes in weather patterns, measurement methods or data quality. These findings create a feedback loop to data collection: new observations are collected, checked and used to evaluate or retrain the model so that it remains reliable over time.

REFLECTION

- *Compare your group's actual use of generative AI with the responsible approach identified before coding.*
- *At which stages did AI assist your process, which decisions remained your group's responsibility, and how did you verify the generated code or explanations?*
- *Identify one improvement you would make to your process in a future activity to better support learning, transparency and academic integrity.*